Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №4
Варіант 14
На тему
«Рекурсія та мемоізація в Python»

Виконав:
Студент групи ФЕП-21с
Хороший Святослав

Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи

Зрозуміти рекурсивне розбиття задач, базові випадки та обмеження стеку. Реалізувати обходи n-арного дерева, застосувати functools.lru_cache і порівняти рекурсивні обчислення з ітеративними альтернативами.

Теоретичні відомості

Рекурсивний алгоритм розв’язує менші підзадачі до досягнення базового випадку. Preorder відвідує вузол перед дітьми, postorder — після дітей. DFS використовує глибину, BFS обходить рівні за допомогою черги. Висота порожнього дерева дорівнює 0, листка — 1.

Мемоізація зберігає результати підзадач. Lru_cache надає cache_info() для статистики та cache_clear() для очищення. CPython має ліміт рекурсивних викликів і не виконує TCO. Кешування не усуває витрати стеку, тому для глибокого дерева доречні власний стек або черга.

Хід роботи

Для варіанта 14 я створив незмінний Node із tuple дітей. Реалізував рекурсивні preorder, postorder, пошук першого значення та висоту. Додав ітеративні DFS, BFS, пошук і висоту. Обходи зберігають порядок дітей, не змінюючи вхідну структуру.

У height_cached я застосував lru_cache. Для Node використано eq=False: ключ кешу спирається на ідентичність вузла й не потребує рекурсивного хешування всього дерева. Повторні запити та спільні незмінні піддерева дають влучання в кеш. Окремі вузли з однаковим вмістом кешуються окремо.

У basics.py я виконав обов’язкову базу з документа: тришляховий quicksort, рекурсивні обходи бінарного дерева, ітеративні BFS/DFS та Fibonaccі з кешем і циклом. Quicksort використовує власний Random(seed), тому глобальний random-стан не змінюється.

У тестах перевірено порожнє дерево, листок, порядок обходів, пошук, висоту, незмінність і API кешу. Ітеративні алгоритми перевірено на ланцюжку з 1500 вузлів без зміни ліміту рекурсії. Quicksort порівнюється із sorted, Fibonaccі — з ітеративною версією.

main.py запускає демонстрацію з app.py. Benchmark порівнює висоту дерева з 1093 вузлів: рекурсію без кешу, ітерацію, холодний і прогрітий кеш. Використано медіану п’яти повторів по 100 операцій. Підготовка дерева не входить у замір; холодний замір включає очищення кешу.

lab04 / algorithms.py

```python
"""Варіант 14: чисті обходи n-арного дерева та кешування висоти."""

from collections import deque
from collections.abc import Iterator
from dataclasses import dataclass
from functools import lru_cache


@dataclass(frozen=True, slots=True, eq=False)
class Node:
    value: int
    children: tuple["Node", ...] = ()

    def __post_init__(self) -> None:
        object.__setattr__(self, "children", tuple(self.children))


def preorder(tree: Node | None) -> Iterator[int]:
    if tree is None:
        return
    yield tree.value
    for child in tree.children:
        yield from preorder(child)


def postorder(tree: Node | None) -> Iterator[int]:
    if tree is None:
        return
    for child in tree.children:
        yield from postorder(child)
    yield tree.value

```

Рис. 1. Файл algorithms.py, рядки 1–32

lab04 / algorithms.py

```python

def find_recursive(tree: Node | None, value: int) -> Node | None:
    if tree is None or tree.value == value:
        return tree
    for child in tree.children:
        found = find_recursive(child, value)
        if found is not None:
            return found
    return None


def dfs_preorder_iter(tree: Node | None) -> list[int]:
    stack, result = ([tree] if tree is not None else []), []
    while stack:
        node = stack.pop()
        result.append(node.value)
        stack.extend(reversed(node.children))
    return result


def bfs_level_order(tree: Node | None) -> list[int]:
    queue, result = deque([tree] if tree is not None else []), []
    while queue:
        node = queue.popleft()
        result.append(node.value)
        queue.extend(node.children)
    return result


def find_iterative(tree: Node | None, value: int) -> Node | None:
    stack = [tree] if tree is not None else []
    while stack:
```

Рис. 2. Файл algorithms.py, рядки 33–64

lab04 / algorithms.py

```
65          node = stack.pop()
66          if node.value == value:
67              return node
68          stack.extend(reversed(node.children))
69      return None
70
71
72  def height_recursive(tree: Node | None) -> int:
73      """Висота в кількості вузлів: порожнє дерево 0, листок 1."""
74      if tree is None:
75          return 0
76      return 1 + max((height_recursive(c) for c in tree.children), default=0)
77
78
79  @lru_cache(maxsize=None)
80  def height_cached(tree: Node | None) -> int:
81      if tree is None:
82          return 0
83      return 1 + max((height_cached(c) for c in tree.children), default=0)
84
85
86  def height_iterative(tree: Node | None) -> int:
87      if tree is None:
88          return 0
89      queue = deque([tree])
90      height = 0
91      while queue:
92          height += 1
93          for _ in range(len(queue)):
94              queue.extend(queue.popleft().children)
95      return height
```

Рис. 3. Файл algorithms.py, рядки 65–95

lab04 / basics.py

```
1   """Обов'язкова база ЛР4: quicksort, бінарне дерево та Fibonacci."""
2
3   from collections import deque
4   from collections.abc import Iterator, Sequence
5   from dataclasses import dataclass
6   from functools import lru_cache
7   from random import Random
8
9
10  def quicksort(values: Sequence[int], *, seed: int = 14) -> list[int]:
11      """Власний RNG із заданим seed: глобальний random-стан не змінюється."""
12      rng = Random(seed)
13
14      def sort(part: Sequence[int]) -> list[int]:
15          if len(part) <= 1:
16              return list(part)
17          pivot = rng.choice(part)
18          left = [x for x in part if x < pivot]
19          middle = [x for x in part if x == pivot]
20          right = [x for x in part if x > pivot]
21          return sort(left) + middle + sort(right)
22
23      return sort(values)
24
25
26  @dataclass(frozen=True, slots=True)
27  class BinaryNode:
28      key: int
29      left: "BinaryNode | None" = None
30      right: "BinaryNode | None" = None
```

Рис. 4. Файл basics.py, рядки 1–30

lab04 / basics.py

```
31
32
33  def preorder(tree: BinaryNode | None) -> Iterator[int]:
34      if tree is None:
35          return
36      yield tree.key
37      yield from preorder(tree.left)
38      yield from preorder(tree.right)
39
40
41  def inorder(tree: BinaryNode | None) -> Iterator[int]:
42      if tree is None:
43          return
44      yield from inorder(tree.left)
45      yield tree.key
46      yield from inorder(tree.right)
47
48
49  def postorder(tree: BinaryNode | None) -> Iterator[int]:
50      if tree is None:
51          return
52      yield from postorder(tree.left)
53      yield from postorder(tree.right)
54      yield tree.key
55
56
57  def bfs_level_order(tree: BinaryNode | None) -> list[int]:
58      queue, result = deque([tree] if tree is not None else []), []
59      while queue:
60          node = queue.popleft()
```

Рис. 5. Файл basics.py, рядки 31–60

lab04 / basics.py

```
61          result.append(node.key)
62          queue.extend(child for child in (node.left, node.right) if child is not None)
63      return result
64
65
66  def dfs_preorder_iter(tree: BinaryNode | None) -> list[int]:
67      stack, result = ([tree] if tree is not None else []), []
68      while stack:
69          node = stack.pop()
70          result.append(node.key)
71          stack.extend(child for child in (node.right, node.left) if child is not None)
72      return result
73
74
75  @lru_cache(maxsize=None)
76  def fib(n: int) -> int:
77      if n < 0:
78          raise ValueError("n має бути невід'ємним")
79      if n < 2:
80          return n
81      return fib(n - 1) + fib(n - 2)
82
83
84  def fib_iter(n: int) -> int:
85      if n < 0:
86          raise ValueError("n має бути невід'ємним")
87      a, b = 0, 1
88      for _ in range(n):
89          a, b = b, a + b
90      return a
```

Рис. 6. Файл basics.py, рядки 61–90

lab04 / app.py

```
"""I/O-демонстрація обходів, кешу та базових алгоритмів."""

import sys

from algorithms import (
    Node,
    bfs_level_order,
    dfs_preorder_iter,
    find_recursive,
    height_cached,
    height_iterative,
    height_recursive,
    postorder,
    preorder,
)
from basics import fib, fib_iter, quicksort


def main() -> None:
    tree = Node(1, (Node(2, (Node(4), Node(5))), Node(3, (Node(6),)), Node(7)))
    print("=== N-АРНЕ ДЕРЕВО | ВАРІАНТ 14 ===")
    print("Preorder:", list(preorder(tree)))
    print("Postorder:", list(postorder(tree)))
```

Рис. 7. Файл app.py, рядки 1–23

lab04 / app.py

```
    print("DFS:", dfs_preorder_iter(tree))
    print("BFS:", bfs_level_order(tree))
    found = find_recursive(tree, 6)
    print("Пошук 6:", found.value if found is not None else None)
    print("Пошук 99:", find_recursive(tree, 99))
    print("Висота (рекурсія / цикл):", height_recursive(tree), height_iterative(tree))
    shared = Node(8, (Node(9), Node(10)))
    dag = Node(0, (shared, shared))
    height_cached.cache_clear()
    print("Висота зі спільним піддеревом:", height_cached(dag))
    print("Кеш після першого виклику:", height_cached.cache_info())
    height_cached(dag)
    print("Кеш після повторного виклику:", height_cached.cache_info())
    height_cached.cache_clear()
    print("Кеш після очищення:", height_cached.cache_info())
    print("Quicksort:", quicksort([3, 1, 2, 2]))
    print("Fibonacci(20), рекурсія / цикл:", fib(20), fib_iter(20))
    fib.cache_clear()
    print("Ліміт рекурсії:", sys.getrecursionlimit())


if __name__ == "__main__":
    main()
```

Рис. 8. Файл app.py, рядки 24–46

lab04 / benchmark.py

```
"""Вимірювання рекурсивної, ітеративної та кешованої висоти."""

from collections.abc import Callable
from statistics import median
from timeit import repeat

from algorithms import Node, height_cached, height_iterative, height_recursive


def make_tree(depth: int, value: int = 0) -> Node:
    if depth == 0:
        return Node(value)
    return Node(value, tuple(make_tree(depth - 1, 3 * value + i + 1) for i in range(3)))


def main() -> None:
    tree = make_tree(6)
    number, repeats = 100, 5

    def cold() -> int:
        height_cached.cache_clear()
        return height_cached(tree)
```

Рис. 9. Файл benchmark.py, рядки 1–22

lab04 / benchmark.py

```

    def measure(fn: Callable[[], int]) -> float:
        return median(repeat(fn, number=number, repeat=repeats)) / number * 1e6

    recursive = measure(lambda: height_recursive(tree))
    iterative = measure(lambda: height_iterative(tree))
    cached_cold = measure(cold)
    height_cached.cache_clear()
    height_cached(tree)
    cached_warm = measure(lambda: height_cached(tree))
    print("Дерево: 1093 вузли, висота 7; 5 повторів по 100 операцій")
    print(f"Рекурсія без кешу: {recursive:.2f} мкс/операцію")
    print(f"Ітеративна висота: {iterative:.2f} мкс/операцію")
    print(f"Холодний кеш (із cache_clear): {cached_cold:.2f} мкс/операцію")
    print(f"Прогрітий кеш: {cached_warm:.2f} мкс/операцію")
    print("Стан прогрітого кешу:", height_cached.cache_info())
    print("Підготовку дерева не включено; значення залежать від середовища.")
    height_cached.cache_clear()


if __name__ == "__main__":
    main()
```

Рис. 10. Файл benchmark.py, рядки 23–44

lab04 / tests/test_algorithms.py

```python
from dataclasses import FrozenInstanceError

import pytest

from algorithms import (
    Node,
    bfs_level_order,
    dfs_preorder_iter,
    find_iterative,
    find_recursive,
    height_cached,
    height_iterative,
    height_recursive,
    postorder,
    preorder,
)


@pytest.fixture
def tree() -> Node:
    return Node(1, (Node(2, (Node(4), Node(5))), Node(3, (Node(6),)), Node(7)))


def test_preorder_and_dfs(tree: Node) -> None:
    expected = [1, 2, 4, 5, 3, 6, 7]
    assert list(preorder(tree)) == expected
    assert dfs_preorder_iter(tree) == expected


def test_postorder_and_bfs(tree: Node) -> None:
    assert list(postorder(tree)) == [4, 5, 2, 6, 3, 7, 1]
    assert bfs_level_order(tree) == [1, 2, 3, 7, 4, 5, 6]

```

Рис. 11. Файл tests/test_algorithms.py, рядки 1–33

lab04 / tests/test_algorithms.py

```python

@pytest.mark.parametrize("value", [1, 2, 4, 6, 7, 99])
def test_search(tree: Node, value: int) -> None:
    recursive = find_recursive(tree, value)
    assert recursive is find_iterative(tree, value)
    assert (recursive is None) == (value == 99)


def test_first_match_in_preorder() -> None:
    first, second = Node(2), Node(2)
    tree = Node(1, (first, second))
    assert find_recursive(tree, 2) is first
    assert find_iterative(tree, 2) is first


def test_empty_and_leaf() -> None:
    assert list(preorder(None)) == list(postorder(None)) == []
    assert bfs_level_order(None) == dfs_preorder_iter(None) == []
    assert find_recursive(None, 1) is find_iterative(None, 1) is None
    assert height_recursive(None) == height_iterative(None) == 0
    assert list(preorder(Node(5))) == list(postorder(Node(5))) == [5]
    assert height_recursive(Node(5)) == height_iterative(Node(5)) == 1


def test_height(tree: Node) -> None:
    height_cached.cache_clear()
    assert height_recursive(tree) == height_cached(tree) == height_iterative(tree) == 3
    height_cached.cache_clear()


def test_cache_shared_subtree_and_clear() -> None:
    shared = Node(2, (Node(3),))
    tree = Node(1, (shared, shared))
```

Рис. 12. Файл tests/test_algorithms.py, рядки 34–66

lab04 / tests/test_algorithms.py

```
67      height_cached.cache_clear()
68      assert height_cached(tree) == 3
69      info = height_cached.cache_info()
70      assert info.hits == 1 and info.misses == 3 and info.currsize == 3
71      assert height_cached(tree) == 3
72      assert height_cached.cache_info().hits == 2
73      height_cached.cache_clear()
74      assert height_cached.cache_info().currsize == 0
75
76
77  def test_no_mutation(tree: Node) -> None:
78      original_children = tree.children
79      original = list(preorder(tree))
80      list(postorder(tree))
81      dfs_preorder_iter(tree)
82      bfs_level_order(tree)
83      find_recursive(tree, 99)
84      height_recursive(tree)
85      assert tree.children is original_children
86      assert list(preorder(tree)) == original
87      with pytest.raises(FrozenInstanceError):
88          tree.value = 99  # type: ignore[misc]
89
90
91  def test_deep_tree_iteratively() -> None:
92      tree = Node(1499)
93      for value in reversed(range(1499)):
94          tree = Node(value, (tree,))
95      assert height_iterative(tree) == 1500
96      assert dfs_preorder_iter(tree) == list(range(1500))
97      assert bfs_level_order(tree) == list(range(1500))
98      assert find_iterative(tree, 1499) is not None
```

Рис. 13. Файл tests/test_algorithms.py, рядки 67–98

lab04 / tests/test_basics.py

```
1   from random import Random
2
3   import pytest
4
5   from basics import (
6       BinaryNode,
7       bfs_level_order,
8       dfs_preorder_iter,
9       fib,
10      fib_iter,
11      inorder,
12      postorder,
13      preorder,
14      quicksort,
15  )
16
17
18  @pytest.mark.parametrize("values", [[], [1], [3, 2, 1], [2, 2, 1, 3, 2], [-1, 0, -3]])
19  def test_quicksort(values: list[int]) -> None:
20      original = list(values)
21      assert quicksort(values) == sorted(values)
22      assert values == original
23
24
25  def test_quicksort_random_and_reproducible_seed() -> None:
26      rng = Random(14)
27      values = [rng.randrange(-100, 101) for _ in range(1000)]
28      assert quicksort(values, seed=14) == sorted(values)
29      assert quicksort(values, seed=42) == sorted(values)
30
```

Рис. 14. Файл tests/test_basics.py, рядки 1–30

lab04 / tests/test_basics.py

```
31
32  def test_binary_traversals() -> None:
33      tree = BinaryNode(
34          4,
35          BinaryNode(2, BinaryNode(1), BinaryNode(3)),
36          BinaryNode(6, None, BinaryNode(7)),
37      )
38      assert list(preorder(tree)) == dfs_preorder_iter(tree) == [4, 2, 1, 3, 6, 7]
39      assert list(inorder(tree)) == [1, 2, 3, 4, 6, 7]
40      assert list(postorder(tree)) == [1, 3, 2, 7, 6, 4]
41      assert bfs_level_order(tree) == [4, 2, 6, 1, 3, 7]
42      assert list(inorder(None)) == bfs_level_order(None) == []
43
44
45  def test_fibonacci_values_and_cache() -> None:
46      fib.cache_clear()
47      assert [fib(n) for n in range(10)] == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
48      assert all(fib(n) == fib_iter(n) for n in range(100))
49      before = fib.cache_info().hits
50      fib(99)
51      assert fib.cache_info().hits == before + 1
52      fib.cache_clear()
53      assert fib.cache_info().currsize == 0
54
55
56  def test_fibonacci_negative() -> None:
57      with pytest.raises(ValueError):
58          fib(-1)
59      with pytest.raises(ValueError):
60          fib_iter(-1)
```

Рис. 15. Файл tests/test_basics.py, рядки 31–60

lab04 / main.py

```
1  """Точка входу лабораторної роботи №4, варіант 14."""
2
3  from app import main
4
5  if __name__ == "__main__":
6      main()
```

Рис. 16. Файл main.py, рядки 1–6

Запуск і результати перевірки

lab04 / Вивід команд терміналу

```
$ python main.py
=== N-АРНЕ ДЕРЕВО | ВАРІАНТ 14 ===
Preorder: [1, 2, 4, 5, 3, 6, 7]
Postorder: [4, 5, 2, 6, 3, 7, 1]
DFS: [1, 2, 4, 5, 3, 6, 7]
BFS: [1, 2, 3, 7, 4, 5, 6]
Пошук 6: 6
Пошук 99: None
Висота (рекурсія / цикл): 3 3
Висота зі спільним піддеревом: 3
Кеш після першого виклику: CacheInfo(hits=1, misses=4, maxsize=None, currsize=4)
Кеш після повторного виклику: CacheInfo(hits=2, misses=4, maxsize=None, currsize=4)
Кеш після очищення: CacheInfo(hits=0, misses=0, maxsize=None, currsize=0)
Quicksort: [1, 2, 2, 3]
Fibonacci(20), рекурсія / цикл: 6765 6765
Ліміт рекурсії: 1000
```

Рис. 17. Запуск демонстрації

lab04 / Вивід команд терміналу

```
$ python -m pytest -q
.......................                                  [100%]
23 passed in 0.02s

$ python -m mypy .
Success: no issues found in 7 source files

$ python -m black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.

$ python -m ruff check .
All checks passed!
```

Рис. 18. Тести та перевірки якості

Порівняння часу обчислення висоти

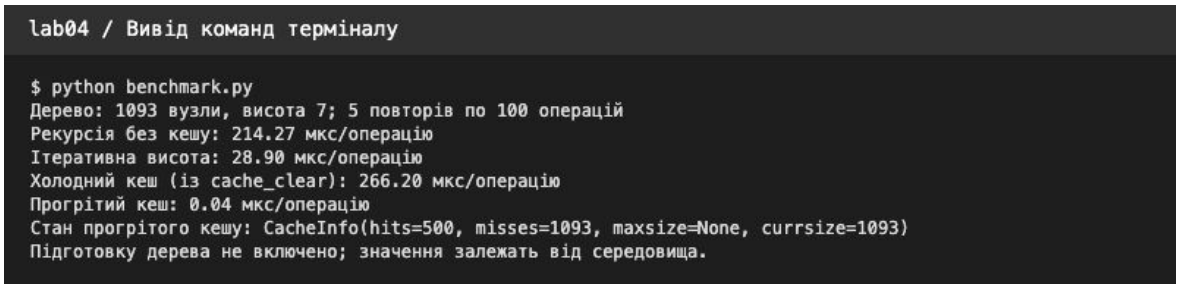


Рис. 19. Порівняння часу операцій

Перший обхід звичайного дерева не має повторних підзадач, тому холодний кеш додає накладні витрати. Прогрітий кеш повертає вже обчислену висоту кореня. Це повторний запит, а не повторний повний обхід; підходи виміряно окремо.

Висновок

Я реалізував рекурсивні та ітеративні алгоритми для n-арного дерева, застосував мемоізацію висоти й продемонстрував статистику та очищення кешу. Додав базові алгоритми з умови. Усі 23 тести пройшли; mypy, black і ruff помилок не виявили.

Рекурсія дає короткий опис обходів, але глибина обмежена стеком CPython. Ітеративні алгоритми працюють із глибокими структурами за допомогою власних контейнерів. Мемоізація корисна для повторних запитів і спільних піддерев, вимагаючи додаткової пам’яті та витрат першого обчислення.

Роботу виконано за Lab_04.docx, варіант 14. Структуру algorithms.py, app.py та tests/test_algorithms.py адаптовано зі зразка github.com/r1tokk/lab04.